An Open-Source Toolbox for Direct and Advanced Interactions with Fuzzy PID Controllers

***Abstract*—Until nowadays, industrial processes have been widely controlled by empirical methods. Fuzzy logic, which by definition is based on such empirical reasoning and expertise, provides a more robust and experimentally validated results compared to conventional PID controllers. An inherent issue is that such expertise is built through practice. It cannot be properly described or taught. It requires two key elements to develop it: a dedicated software tool and an experimentation environment, whether physical or simulated. Existing software solutions such as MATLAB's Fuzzy Toolbox are time- and energy-consuming, as the literature lacks of ready-to-use tools for this purpose. Therefore, this paper introduces P4FPID, a novel open-source fuzzy logic toolbox for direct and smooth fuzzy PID control relying mainly on human intuition and expertise. Developed using Processing 4.4, P4FPID adopts innovative graphical user interface interaction dynamics that enable practitioners to quickly and easily configure and experiment with fuzzy PID controllers in real time via simulations or hardware integration. Thanks to its dynamic simulation model, P4FPID allows users to immediately observe system feedback. Unlike existing solutions, the proposed software supports fuzzy set subnormality exploration, and integrates an interactive genetic algorithm capable of animating more than 20 different optimization scenarios. Moreover, P4FPID supports industrial PLCs (IEC 61131) and MATLAB file formats, making it suitable for both professional and academic use. A benchmark experiment demonstrated that P4FPID outperforms its equivalent MATLAB Simulink blocks in execution time, achieving speed gains averaging up to 130× faster.**

Note to Practitioners—The core foundation of fuzzy logic control in industrial automation relies on modeling human expertise, which is best achieved through trials and experiments. Existing fuzzy logic software tools are energy- and time-consuming and lack several important features required for a feasible practice of fuzzy logic control. Therefore, this paper introduces a novel open-source Processing 4.4 toolbox called P4FPID, which enables practitioners to design and tune fuzzy PID controllers through direct and advanced real-time interaction. Unlike existing solutions, P4FPID provides engineers with the ability to immediately observe the system's feedback during simulation, learn the sequence of fuzzy PID control through an infinite-time animation mode, and directly interact with fuzzy controllers during real-life hardware plugging. All these features are supported by innovative systematic tuning patterns of the fuzzy system. In addition, P4FPID integrates a flexible genetic algorithm that can handle more than 20 optimization scenarios using step-by-step animations. The toolbox is open source for broader development. It supports exporting industrial PLC (IEC 61131) and MATLAB file formats, making it suitable for both professional and academic use. The software has been benchmarked against MATLAB's equivalent simulation blocks, where results showed that P4FPID outperforms them with an average speed gain of up to 130 times.

***Index Terms*—Fuzzy logic toolbox, Control theory, Automation, Fuzzy PID, Computer-aided engineering, Computer-aided learning**

I. Introduction

In control theory and automation domains, a common complaint about fuzzy logic control is the complexity and time consumption [1]. This can potentially repulse users who are interested in fuzzy logic from implementing this robust control technique despite its proven efficiency in different case studies [2]. It returns to the fact that the Fuzzy Inference System (FIS) adopts an important number of Membership Functions (MFs) of different types and position probabilities, fuzzy rules (which can be important in terms of amount and choices), different implication laws, and defuzzification methods.

It is obvious that reading scientific articles or books about implementing fuzzy logic is not enough to build fuzzy control expertise. The majority of state-of-the-art implementation of fuzzy control didn't rely on trial or experiments as it meant to be. Instead, they focused on optimization techniques like adopting pseudo-random algorithms [3][4] to tun the fuzzy controller. On theory it is valid, but in practice it is not as optimum as claimed. Because it relies on the given mathematical model which can not reflect the entire dynamics of the real-system. There are always be fuzzy parts of the system which is hard to be captured analytically. Indeed, this particularity (the system's uncertainty) is where fuzzy logic control robustness is needed the most in which it requires experience and direct interactions with the controlled system.

Based on the foundation of this technique (which is a mathematical modeling of human expertise), the optimal solution in our view is conducting real-life practical experimentation of fuzzy systems configuration, whether using simulators or real-time hardware implementations. Now, concerning fuzzy logic software tools like [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], they focus more on their products' functionality than providing the user the tools he needs to master fuzzy logic control. We believe that they lack a number of crucial features. For instance, providing the user the ability to perform systematic real-time modifications. Another important aspect is that they do not integrate training environments that allow users to promptly observe and learn from system feedback (without waiting for a fair amount of time to see the result of a small modification of the FIS parameters). Also, for better understanding, visualizing the sequence of fuzzy control rule firing and the corresponding system evolution is required for future development of expert based control tunings.

Therefore, this paper presents an open-source application-oriented toolbox named P4FPID (Processing 4.4 Fuzzy PID). It has been developed to provide more advanced and time/energy cost interactions with fuzzy PID controllers. The software

is open-source and subject to further development of more complex or specific exploitation. For interoperability matters, P4FPID provides IEC 61131 [15] format toward integration with industrial PLCs, JFML (Java Fuzzy Markup Language) for software interoperability, as well as MATLAB-compatible .fis files for broader research applications. To the best of our knowledge, this work's particularity is the first of its genre as distinguished from state-of-the-art solutions [5], [6], [7], [8], [9], [10], [11]. Technically, what distinguishes P4FPID from the aforementioned are these main features that come as follows:

- Emphasizes on human expertise by adopting innovative GUI's interaction dynamics that assists the user applying various and intuitive tuning actions.
- Runs in three modes: dynamic simulation, infinite-time animation, and real-time control (hardware integration), enabling the user to apply the aforementioned interactions during each mode.
- Offers a flexible Genetic Algorithm (GA) that can animate step by step more than 20 optimization scenarios.
- Outperforms the equivalent MATLAB Simulink blocks, achieving on average a 130 fold increase in execution time.

Additionally, from the control theory perspective, P4FPID enables the experimentation of: fuzzy sets subnormality, and introduces a new flexible fuzzy MF called Double-Edged Gaussian MF. The rest of the paper is organized as follows: The second Section sheds light on the state-of-the-art software of fuzzy logic. In the third Section, we talk about the used programming IDE (Integrated Development Environment) and the overall architecture of P4FPID. Section 4 discusses solely the provided advantages of P4FPID and their importance for an accurate expert-based use of fuzzy PID control. A benchmark experiment is detailed in fifth Section between P4FPID and equivalent MATALB Simulink blocks. Section 6 highlights P4FPID current version application scope and limitations. Finally, the paper ends with a conclusion in the last Section.

II. Related Works:

Jesus Alcala-Fdez et al [13] published a review article where the authors classified the state-of-the-art fuzzy logic software in terms of characteristics (suites, toolbox, library, and code) and purposes (general purpose, application-oriented, and fuzzy markup languages). Based on the nature of our contribution, we only focus on fuzzy logic toolboxes and suites that can be used for fuzzy logic control in automation and control theory used for training or teaching purposes. The rest are process-based application of fuzzy logic control (implementation of fuzzy logic control without focus on the used software tool for that).

A. Fuzzy logic suites and toolboxes

The first one is the well-known Fuzzy Logic Toolbox of MATLAB FTS (Fuzzy Tool System), owned by MathWorks [5]. MATLAB's fuzzy tool is designed to create scalable input/output fuzzy systems. Using the MATLAB IDE, Laurent Foulloy [11] later introduced an educational library called FlouLib, aimed at providing a clearer graphical representation of fuzzy systems. FlouLib makes building programming blocks that scale the use of the FLT modules better connected and presented with each other and other simulation blocks using a sophisticated GUI. After the growing demand for fuzzy interval type-2 software, also using the MATLAB IDE, several works provided a toolbox for type-2 fuzzy logic [16], [10]. The authors dealt with the necessities that come with the implementation of the mentioned technique. However, they are always limited to the same design and architecture of the original MATLAB FLT. Jailsingh Bhookya et al. [2] developed a basic GUI for controlling the water level of a process trainer toolkit. The user uses the GUI to send 3 gain factor values while the executing logic is done in MATLAB FLT. Now, away from MATLAB, FuzzyTECH [6] is optimized for industrial utilization such as automation and robotics. Mohd Shahrieel Mohd Aras presented a comparison of both MATLAB FLT and FuzzyTECH in his study [17]. QtFuzzy [9] is a general-purpose C++ fuzzy logic suite built using the open-source fuzzylite C++ library [18], developed by the same author. It offers additional features, but it's still less intuitive compared to MATLAB's Fuzzy Logic Toolbox and FuzzyTECH when it comes to GUI manipulation from the point of view of dragging MFs parameters which we consider as an important aspect. Up to this point, all the mentioned suites are commercial software. On the other hand, two free alternatives, Xfuzzy (Java-based) and FisPro (C++-based), have been proposed in [8] and [19] respectively. Notably, Xfuzzy supports the extraction of various file formats, while FisPro allows the implementation of gradual implicative rules [13].

B. Fuzzy logic libraries

Fuzzy logic libraries exist in a sufficient number of different programming languages, like Python, C++, C#, R, Arduino, and Java [12]. Historically speaking, JFuzzyLogic (Java library) [20] is the first library that formally adheres to the IEC 61131 standard [15], which makes it readable by industrial PLCs (Programmable Logic Controllers), as well as control applications. It has an Eclipse plugin module to visualize and test the written code output and built-in deterministic optimization algorithms. Sponsored by the IEEE Computational Intelligence Society, a new markup language named JFML (Java Fuzzy Markup Language) was agreed upon under IEEE Std 1855-2016. It generates an XML-based language file [14] for wider software interoperability. The authors offered a complete website [21] to unify fuzzy logic implementation formats to be widely adopted by both the professional and the academic communities. Authors in [22] proposed UPAFuzzySystems; a Python library for control systems applications. UPAFuzzySystems is a clone of jFuzzyLogic in Python programming language. The authors relied on the pre-existing control library to provide a sophisticated environment and examples for users to easily program their own simulations experiments which by the way still demand programming practitioners to do the job of simulations.

C. Discussion

To be fair, selecting a peer competitive software (toolbox or suite) to P4FPID among the mentioned works is not fully applicable. The mentioned suites and libraries focus on designing general purpose fuzzy systems, flexible programming architectures, multiple input-output systems.. etc to be used for any genre of applications. To the best of our knowledge, non of the works in the-state-of-the-art includes simulators (which by the way implies the existing of a specific application domain). Aside from that, they also do not offer advanced interactions, animation, or optimization like the ones that exist in our solution. Based on that, the nearest equivalent-software for P4FPID is MATLAB Simulink model with integrated fuzzy controller as shown in Figures 4 and 5.

III. P4FPID using Processing

Processing is an open-source programming language and development environment mainly designed for education, visual arts, and creative coding [23]. However, developing a GUI-driven applications toolbox in a loop-based programming scheme like Processing presents unique challenges due to the lack of tools that facilitate this task compared to the available ones in C#, C++, or Python. On the other hand, using Processing provides a better control over making customized creative animations and drawings. Based on the fact that P4FPID is application-oriented (fixed structure of fuzzy PID controllers of 2 inputs and a single output), it has been developed based on straightforward coding manner (from scratch). For these reasons, P4FPID's coding/development is fully original (library-free) except using Serial Communication Library, as no peer-reviewed fuzzy logic library covered this IDE specifically. P4FPID's source code will be publicly available under the GNU General Public License v3.0 via a public GitHub repository for further development and implementations upon the paper's publication.

The source code is optimized in a way that all heavy load (control & fuzzy logic) calculations run in independent threads, which by the way invokes additional complexity of variable/events entanglement and scheduling as shown in Figure 1. A user guide is provided alongside the P4FPID toolbox. About the software's validation, the software has been extensively tested locally and in students' practical work sessions, demonstrating reliable outputs and stable operating (See software evaluation in supplementary files). Moreover, the product is an open-source software opened for further development and customization by the fuzzy logic community.

IV. P4FPID Features

This version of P4FPID is based on Mamdani inference system using Type-1 fuzzy sets. A screenshot of one of P4FPID's interfaces is presented in Figure 2. In this section, we only mention the essential contributions compared with the relevant works in the state-of-the-art, especially the fuzzy tool of MATLAB, as it is one of the most used toolboxes in the academic field and the only competitor that can be used for systems' simulations (MATALB Simulink).

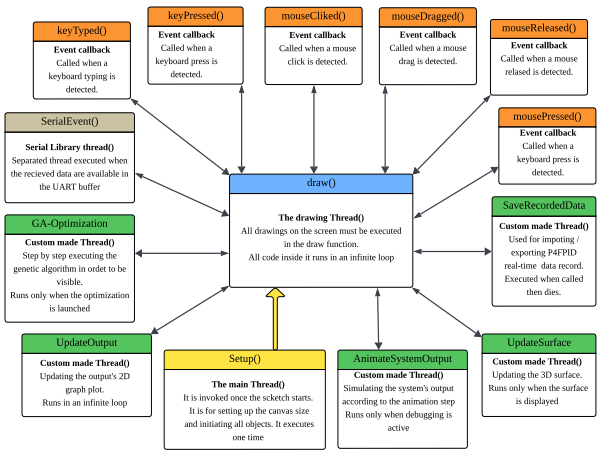

Fig. 1: P4FPID Programming Blocks Architecture

A. Membership Functions Manipulation

Existing state-of-the-art fuzzy-logic software/suites [5], [6], [7], [8], [9], [10], [11], [18], [14], [20] generally treat MFs design as a static. MATLAB fuzzy toolbox is an exception. It offers users the hand to manipulate MF positions directly using the computer mouse (a crucial feature for intuitive geometric adjustments). But even though, its interactions remain limited lacking dynamic behavioral coupling.

In contrast, P4FPID embeds a series of intuitive human–machine interaction dynamics that make the design process exploratory, responsive, and cognitively aligned with expert visualization. The intuitive addition or subtraction of MFs with automatic rule-table generation allows practitioners to instantly observe the logical consequences of their conceptual adjustments. The constrained MFs mechanism prevents meaningless configurations by ensuring continuous MF entanglement, thereby guiding the user to construct distributions that reflect real human intuition rather than arbitrary geometries. Similarly, the symmetry option and dynamic push/pull controls provide rapid, physically meaningful manipulations of MFs across the universe of discourse, encouraging experimentation without losing structural coherence. By enabling direct control of MF normality or subnormality [24], P4FPID gives the user the hand to explore this feature and its effect with the aid of organized distributions (ascending or descending order from and to the centered MF). The random MF generator stimulates exposes the sensitivity of the system. Moreover, P4FPID extends this intuitive interaction by allowing the user to widen or narrow the upper and lower widths of MFs simply by scrolling the mouse wheel, offering fine-grained control over the linguistic variable ranges and enabling rapid experimentation with their influence on the system's behavior.

Based on the mentioned MFs in [5], [6], [7], [8], [9], [10], [11], [18], [14], [20], in this paper, we introduce a new type of MF named Double Edged Gaussian (DE-Gaussian, See Figure 3). It is the same as the asymmetrical Gaussian MF but with cut ends. It has been made to be displayed as the triangular MF with 3 parameters $[a_1, a_2, a_3]$. Equation 1 shows how the

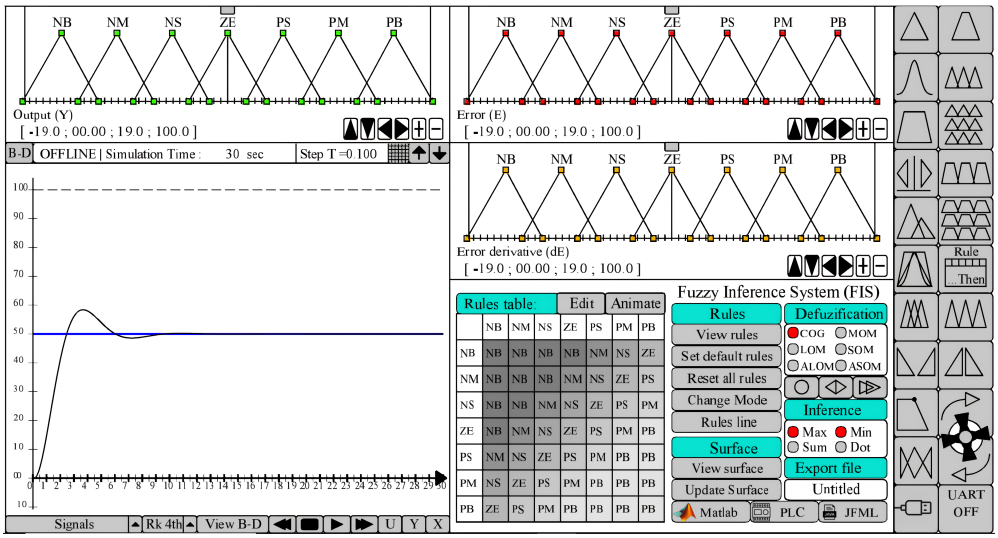

Fig. 2: P4FPID Screenshot

DE-Gaussian MF is centered in a_2 and grounded (near to the X axis but does not cross it) at a_1 and a_3. Doing so gives the user more flexibility compared to the asymmetrical Gaussian MF of MATLAB FTS. He can play with the standard deviation in 2 different ways on both sides. Also, it is time cost-effective when it comes to the defuzzification calculus.

$$\begin{cases} f(x) & = \exp\left(-\frac{(x-a_2)^2}{2\sigma^2}\right) \\ \sigma_{Left} & = \frac{(a_2-a_1)}{C} \\ \sigma_{Right} & = \frac{(a_3-a_2)}{C} \\ C > 1 \end{cases} \quad (1)$$

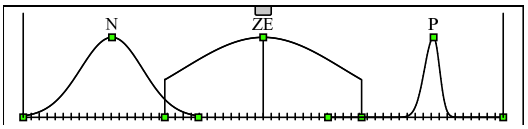

Fig. 3: Double Edged Gaussian Membership Function

Collectively, these interaction dynamics convert fuzzy PID controllers' MF design to experiential process where the practitioner can see, feel, and refine the reasoning structure in real time.

B. FIS Settings

In addition to MFs manipulations, P4FPID redefines how fuzzy rules are perceived and adjusted. The fuzzy rule table is displayed in a layout that enhances the visualization of extreme left/right rules and their corresponding weights, allowing practitioners to better interpret control signal tendencies. A set of built-in algorithms assists the user to rapidly configure the rule base, provide complementary options such as automatic rule configuration [25], full rule/operator reset, and standardized MF nomenclature, ensuring a fine clarity and reproducibility. In a dynamic mode, users can intuitively regulate the rule-flow orientation according to system behavior either (1) maximizing control signals by favoring extreme rules for slow-response systems, or (2) enhancing precision by emphasizing near-zero rules for sensitive systems. Furthermore, unlike state-of-the-art approaches in [5], [6], [26], [9], [7], [6], P4FPID improves rule-selection dynamics through adding a mouse-wheel-based scrolling, allowing practitioners to preview and select "IF...THEN" rules, rule weights, and connectors in a smoother and more responsive manner than traditional interfaces.

C. Systems' Simulation/Animation

The simulation interface is mainly for setting the simulation setups, like choosing the transfer function, the time domain, or the controller type (PID [27] or PD + I [28]) as shown in Figures 4 and 5 (P4FPID contains the discretization time domain image of each controller). To plot the system's output, a flexible architecture has been used to dynamically switch between numerical methods. More details are provided in the user guide document regarding system's plotting and transfer/Z-functions.
As shown in Figures 4 and 5, each gain control factor is followed by a saturation block and another gain factor to automatically normalize each signal before entering and after leaving the fuzzy controller, with respect to the set MFs' range ($Range_E, Range_{dE}$, and $Range_Y$) and the systematic saturations (E_{Max}, dE_{Max}, and Y_{Max}).

Unlike the existing software which the set the ranges as static objects, the user can experiment their effects on the fuzzy rule using by dragging them using the computer mouse. Moreover, he can simulate customized nonlinear systems by following the same logic in the software source code of numerical method programming. However, thanks to the adopted dynamic simulation model, P4FPID provides the following advantages: The trainee is able to Observe the system's output immediately after applying any possible changes in the FIS or the system parameters. Also, he can Launch an infinite-time animation and have the ability to observe the sequence of fuzzy

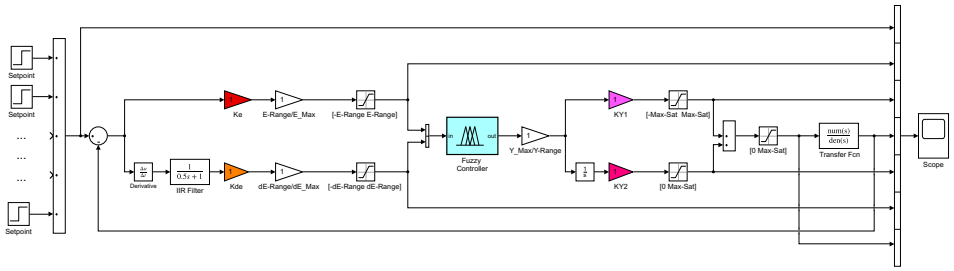

Fig. 4: Control Diagram OF Fuzzy PID Controller [27]

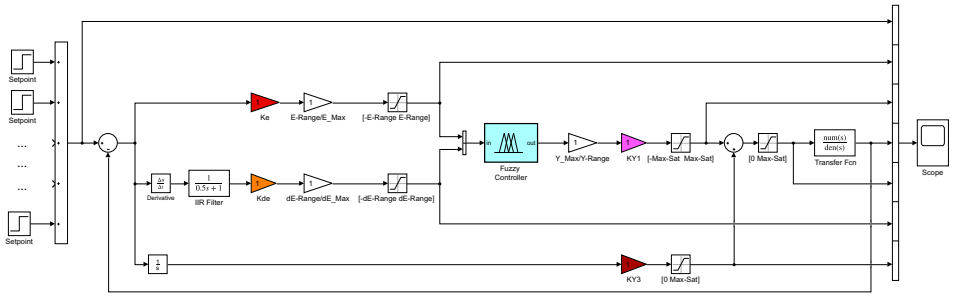

Fig. 5: Control Diagram OF Fuzzy PD + I Controller [28]

control and adjust the controller parameters on-the-fly. He can also change the system's transfer function at any time desired, in which the new transfer function is assigned after the Runge-Kutta algorithm (S domain) / or the recurrence (Z-transform) algorithm finishes the cycle of calculating the current internal states.

D. Controller Optimization

In case the manual tuning was not successful, or for the purpose of teaching nature-inspired optimization to students/-trainees, P4FPID introduces an interactive genetic algorithm (GA). The chosen optimization algorithm (GA [29]) was an arbitrary choice. It has been developed to be as complete as possible in terms of optimization variables and patterns. It optimizes 4 parameters: the MFs positions, the control gain, the MFs normality, and the fuzzy rules table.

The trainee has the option of real-time animation, where he can control the speed of the optimization process and observe the evolution of the fuzzy system. It offers the student 4 types of errors $E(X)$ to be implemented: Mean Squared Error (MSE), Integral of Absolute Error (IAE), Integral of Time-weighted Absolute Error (ITAE), Root Mean Squared Error (RMSE). The fitness function f can be set in two different ways as follows:

- First form:

$$\text{f(x)} = \frac{K}{E(x)+1} + 1$$

- Second form:

$$\text{f(x)} = K.S - E(x);\ S = 2 * \text{Max-Saturation}$$

The user can amplify the chosen fitness function in a polynomial or an exponential way. We made a programming architecture that acts like an attribute as shown in the matrix of Equation 2. This architecture can theoretically have an n variable (n columns), each variable can have a different size, and m number of individuals for each variable. Each row of the GA attribute results in its own fitness score Fit_k based on the chosen fitness function, depending on the optimization case.

$$Atrribute = \begin{bmatrix} V_{1,1} & V_{1,2} & \dots & V_{1,n} \\ V_{2,1} & V_{2,2} & \dots & V_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ V_{m,1} & V_{m,2} & \dots & V_{m,n} \end{bmatrix} \implies \begin{bmatrix} \text{Fit}_1 \\ \text{Fit}_2 \\ \vdots \\ \text{Fit}_m \end{bmatrix} \tag{2}$$

All attribute variables evolve simultaneously, doing inter-crossover with the same vector columns (same variable family, type, and size). Table I summarizes the number of possible optimization scenarios that can be covered using P4FPID (20 optimization scenarios without generalizing the symmetry option to all mentioned scenarios). In the coming subsections, we elaborate more by showing the optimization method of each variable as follows.

1) Membership functions optimization: P4FPID offers the user the ability to choose the variable he intends to optimize. Furthermore, P4FPID's user can choose the side of the optimization (Left or Right side). By having 3 MF variables (the error, the error derivative, and the output), adding the side option increases the total number of optimization variables to 6. The element of the optimization vector represents the position of a single parameter of an MF. Since MFs must be grouped in a specific pattern, this imposes a constrained optimization problem. The developed GA is designed to optimize different MF shapes, allowing for optimization vectors of varying sizes within the same MF set, as already mentioned. This flexibility enables it to handle multiple MF types rather

TABLE I: Main Optimization Scenarios

MFs partitions
MVB
MVB side (left/right side)
Random MFS partition
Linked MFs
Edge MFs
Lower width
Upper width
Symmetrical/Asymmetrical MFs
Unified/Random MFs shapes
MFs Normality Weights (NW)
MVB
MVB side (left/right side)
Toward the center distribution
Toward the edges distribution
Symmetrical / Asymmetrical NW distribution
Control gains
Concerned gain factors
Rules table
(Rules &/or Operators &/or rules weights)
Symmetrical/Asymmetrical Table

than being restricted to a single shape.
The user can optimize the MFs' positions with 4 different patterns, all under the symmetry option. The ones worth detailing in this article are the following:
- Lower width: This feature optimizes the MFs' lower width (base length). The element in the optimization vector contains the percentage $P_{\%}$ (from 0% to 100%). In the first step, the algorithm saves the MFs configuration found by the user to use it as a reference, invoked each time a new individual (optimization vector) is generated. The assignment process is done in ascending order, from the first to the last MF of the MVB. This makes the relevant distance (the distance between the corresponding MF and its left neighbor) vary, not remain fixed for each assignment step. This is why we set percentages in the first place, because the absolute distance depends on the set percentage of the left neighboring MF.
To maintain a reasonable configuration of the MFs without falling into a non-intuitive distribution, we made two assignment constraints, as pictured in Figure 6 and Equation 4.

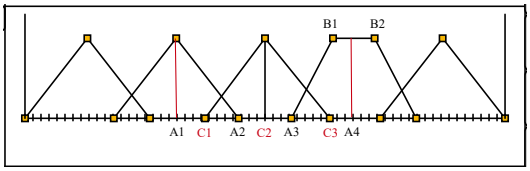

Fig. 6: Lower Width MFs optimization illustration

$$\begin{cases} A_4 = \frac{B_1+B_2}{2} \\ C_1 = A_1 + P_{\%} \times (min(A_2, C_2) - A_1) \\ C_3 = A_3 + P_{\%} \times (max(A_3, C_2) - A_3) \end{cases} \quad (3)$$

-Upper width: Using the same principle as the Lower width optimization, this feature is designed to optimize only two types of MFs: trapezoidal and DE-Gaussian. It adjusts the width of these MFs, where each element of the optimization vector represents a percentage ranging from 0% to 100%. For the trapezoidal MF, the optimization adjusts the position of the middle heads, transforming the shape from a full rectangle to a triangle. The assigned percentage is applied symmetrically to both sides, modifying the middle heads toward the right and left. See Figure 7 and Equation 4.
For the edge-Gaussian MF, the optimization varies its shape from a narrow to a wide Gaussian, ranging from $C = 1$ to $C = 10$. $C = 10$ is set by the author as the narrowest possible degree; see Equation 1.

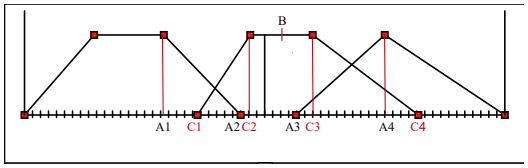

Fig. 7: Upper Width MFs optimization illustration

$$\begin{cases} B = \frac{C_2+C_3}{2} \\ C_2 = B - P_{\%} \times (B - max(A_1, A_2)) \\ C_3 = B + P_{\%} \times (min(A_3, A_4) - B) \end{cases} \quad (4)$$

-Symmetrical MFs: This option enables a symmetrical configuration of the MFs. It applies to both MFs' positions and normality degrees. This option can also be generalized to all previously mentioned features.

2) MFs Normality Optimization: Similar to MF optimization, the user has the same options for optimizing MFs' normality degrees in terms of their number (concerned MFs), sides, and symmetry. Each element of the optimization vector represents a percentage to be assigned as the rule weight for a given MF. Specifically, in MF normality optimization, the user can choose an optimization pattern that pushes the MFs' normality weights toward the center or the edges of the universe of discourse.

3) Control Gains optimization: The user can specify the desired gains and saturations to be optimized along with their respective ranges using one single optimization vector.

4) Rules table optimization: The user has 6 possible optimization scenarios to optimize the rules table as shown in Table I, with the choice of generating a symmetrical table. The optimization vector consists of integer values representing the corresponding number of output MF rule, output MF rule's weight, or/and logical connector. We set the GA attribute to use the rows of the rules or operators table/tables as input optimization vectors. This approach increases the diversity of individuals, aiming to accelerate the search for an optimal solution. For instance, when optimizing all rules, rules' weights, and operators. All optimization vectors are placed side by side to be optimized simultaneously within the attribute as shown in Equation 2.

E. Real-time Control

P4FPID's user can interact with the plugged in system and apply every possible change in the FIS system and observe its corresponding effect on the output signal and the system response. This important feature doesn't exist in MATLAB FTS GUI, which is primarily designed for designing, editing, and analyzing fuzzy inference systems (FIS) in an offline manner. For enhanced visualization in training and teaching contexts, the user can design and link multiple system animations, approximating a Hardware-in-the-Loop (HiL) configuration. See the user guide manual, also embedded within the source code as demonstration demos.

The used communication protocol for that is UART (Universal Asynchronous Receiver Transmitter) using Processing Event handler (*serialEvent()*). In case of running the obtained fuzzy logic controller algorithm inside a microcontroller like Arduino, PIC16F XXX, or PIC18F XXX, it cannot be realized at all times. The limited capabilities of these microcontrollers cannot handle the heavy computational load. Instead of connecting to a third party machine like in [30], P4FPID generates a 3D surface output with different resolutions as a compensatory approach. The user just maps the relevant error and error derivative in the matrix to obtain the corresponding system output value.

V. Benchmark Experiment

A benchmark has been performed to compare the execution speed of both P4FPID and its equivalent simulation blocks in Figures 4 and 5 using two random systems for each time domain as shown in Equations 5 and 6.

$$\text{(Sys 1):} \quad G_1(s) = \frac{s+1}{s^3+s^2+s+1} \tag{5}$$

$$\text{(Sys 2):} \quad G_2(z) = \frac{0.509}{z^{-2}-1.1314z^{-1}+0.64} \tag{6}$$

The experiment was conducted using an Intel CPU i7-9850H (10865 benchmark score from www.cpubenchmark.net) with 16 GB of RAM. Table II shows the used simulation settings.

TABLE II: Simulation Settings

Parameter	Value
Universe of discourse (UoD)	[-100 100]
Display Ranges	[-99 99]
Output discretization points	{2000 , 4000}
Simulation Sampling Time	0.02 second
Defuzzification	Center of gravity
Inference Engine	max/min
MFs type	Trapezoidal
Simulation Engine	Range Kutta 4^{th}
Controller Type	PID (Figure 3)
Step Input signal	50

Our concern doing this benchmark is highlighting the existing differences in the execution time between the two software aiming on empirical implementation of fuzzy logic control of uncertain systems. In this study, we focus on the number on simulation steps. Therefore, we chose nonequivalent number of discretization points; 10 for MATLAB, and 1000 for P4FPID. As long as the resulting outputs exhibit acceptable differences under identical FIS configurations, the comparison remains valid despite the fact that it disfavourates our solution in terms of execution time. Further details regarding the experimental setup are provided in the Supplementary Materials (Software Evaluation document, .fis files, and Simulink models).

TABLE III: Benchmark Experiment

System	MFs	MATLAB	P4FPID	Speed-Up
Sys 1 (5)	$3\times3 \mid 3$	14231.4 ms	82 ms	173.6
	$5\times5 \mid 5$	15393.3 ms	79 ms	187.7
	$7\times7 \mid 7$	18006.2 ms	80 ms	219.6
	$9\times9 \mid 9$	19554.8 ms	76 ms	238.5
Sys 2 (6)	$3\times3 \mid 3$	5257.5 ms	75 ms	64.1
	$5\times5 \mid 5$	5241.5 ms	79 ms	63.9
	$7\times7 \mid 7$	5492.3 ms	79 ms	67.0
	$9\times9 \mid 9$	6009.5 ms	77 ms	73.3

Based on the recorded results in Table III, we clearly see how P4FPID outperforms MATLAB simulation blocks in execution time. As the results shows, the difference is remarkable in the continuous time more than the discrete time domain. In both cases, on average, P4FPID outperforms them with a speed gain around 240 times faster for the continuous time and 70 times faster for the discrete time (a total 130 times faster on average). On the other hand, P4FPID doesn't show a significant difference in speed between the two time domains in which it is averaging around 75 ms execution time.These results are simply attributed to the inherent advantages of using an application-oriented software versus a general-purpose one. MATLAB Simulink performs additional layers of computation, covering a wider spectrum of application scenarios. It is a sophisticated simulation environment designed to manage large-scale experiments, providing reliable and high-precision computational outputs using its internal clock. On the other hand, the mentioned features are not present in our software. More importantly, we note that this benchmark comparison did not consider the advantages of using the aforementioned P4FPID's systematic interactions and features. Obviously, it takes significantly more time and effort for the practitioner to copy the P4FPID's control tunings using MATALB Simulink hence this latter is primarily oriented toward general analytical and synthesis aspects wider than just only fuzzy-control.

VI. P4FPID's Functionality Scope

This current version of P4FPID is made to master fuzzy logic control in a practical & empirical manner (expert-based), mainly targeting (SISO) systems of industrial processes in which the majority of cases are linearized around an operating point. Because each nonlinear system needs a customized simulation block which can be easily achieved using the provided source code to animate a range of nonlinear systems. However,

the user can use this standard version for real-time hardware control (linear and nonlinear SISO systems), or, for example, set up experiments of lookup table records in the case of multi-model system control. He can incorporate the obtained P4FPID controller by extracting the corresponding files to be integrated with industrial PLCs or MATLAB Simulink. Figure 8, portraits of the software functionality.

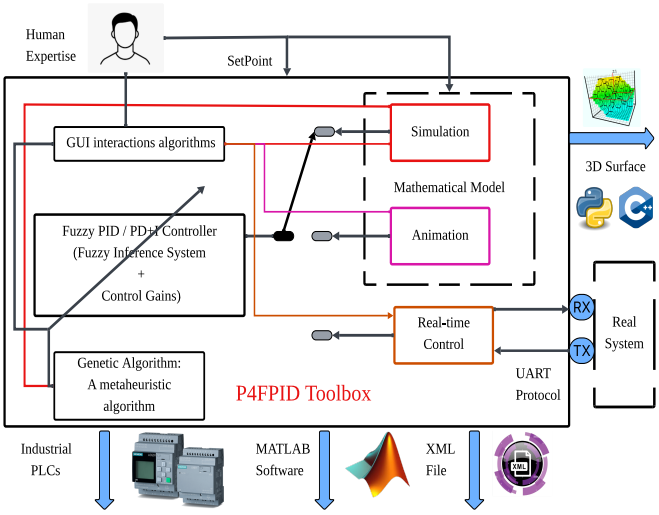

Fig. 8: P4FPID functionality block

VII. Conclusion

This paper presented an open-source fuzzy logic toolbox called P4FPID. The main contribution of this paper resides in providing an open-source software that highlights a new design of GUI-based interaction dynamics attached to a dynamic simulation model for accelerated experimentation of fuzzy logic control. P4FPID facilitates human expertise trials, the feature that has been neglected in many works in the state-of-the-art and substituted with numerous nature-inspired optimization algorithms. Taking MATLAB's Fuzzy Toolbox & Simulink block as a main competitor model, we showed how P4FPID's design and algorithms provide a number of advantages with regard to simulation mode, GUI interaction, and real-time control. We followed with a benchmarking experiment with this latter, where the results showed an important difference between the two software in terms of execution speed, up to 200 times faster on average for continuous time systems and 70 times faster for discrete time simulations. A detailed description of P4FPID's integrated GA, which can animate more than 20 different optimization scenarios, has been provided in this paper. Our solution provides IEC 61131 format for industrial utilization, JFML format file, as well as a ".fis" MATLAB export for broader research use. Overall, in this paper, we discussed P4FPID's programming language and architecture, main features, a benchmarking experiment, and limitations. P4FPID is open-source software, documented and has been used in lectures and practical work classes, where it gained the students' total consensus. The P4FPID is developed as an open-source software that can be easily extended to wider application spectrum in automation and control theory covering different aspects and specifications of fuzzy logic control.

References

[1] F. P. Nishanth *et al.*, "Critical study of type-2 fuzzy logic control from theory to applications," *Heliyon*, vol. 10, no. 8, p. e3516, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772671124003516

[2] J. Bhookya, M. V. Kumar, J. R. Kumar, and A. S. Rao, "Implementation of pid controller for liquid level system using mgwo and integration of iot application," *Journal of Industrial Information Integration*, vol. 28, 2022.

[3] W. Zeng, Q. Jiang, Y. Liu, S. Yan, G. Zhang, T. Yu, and J. Xie, "Core power control of a space nuclear reactor based on a nonlinear model and fuzzy-pid controller," *Progress in Nuclear Energy*, vol. 132, 2021.

[4] K. Premkumar and B. V. Manikandan, "Bat algorithm optimized fuzzy pd based speed controller for brushless direct current motor," *Engineering Science and Technology, an International Journal*, vol. 19, 2016.

[5] L. O. Hall and R. J. Hathaway, "Software review - fuzzy systems toolbox - fuzzy logic toolbox," *IEEE Transactions on Fuzzy Systems*, vol. 4, 1996.

[6] INFORM GmbH, *FuzzyTECH*, 2025, development tool for Fuzzy Logic systems. [Online]. Available: https://fuzzytech.software.informer.com/

[7] F. D. Team, "Fispro: Fuzzy inference system design and optimization," 2025, accessed: 2025-03-25. [Online]. Available: https://www.fispro.org/en/

[8] I. de Microelectrónica de Sevilla, "Xfuzzy: The fuzzy system development environment," 2025, accessed: 2025-03-25. [Online]. Available: https://www2.imse-cnm.csic.es/Xfuzzy/

[9] J. Rada-Vilela, "Qtfuzzylite - a fuzzy logic control library in c++," 2025, accessed: 2025-03-25. [Online]. Available: https://fuzzylite.com/QtFuzzyLite/

[10] M. B. Ozek and Z. H. Akpolat, "A software tool: Type-2 fuzzy logic toolbox," *Computer Applications in Engineering Education*, vol. 16, 2008.

[11] L. Foulloy, R. Boukezzoula, and S. Galichet, "An educational tool for fuzzy control," *IEEE Transactions on Fuzzy Systems*, vol. 14, 2006.

[12] F. A. Pontes, E. Curry, and M. Schukat, "A comparative study of open-source fuzzy modelling toolkit licenses and features," in *IEEE International Conference on Fuzzy Systems*, 2023.

[13] J. Alcala-Fdez and J. M. Alonso, "A survey of fuzzy systems software: Taxonomy, current research trends, and prospects," *IEEE Transactions on Fuzzy Systems*, vol. 24, 2016.

[14] J. M. Soto-Hidalgo, J. M. Alonso, G. Acampora, and J. Alcala-Fdez, "Jfml: A java library to design fuzzy logic systems according to the ieee std 1855-2016," *IEEE Access*, vol. 6, 2018.

[15] A. Otto and K. Hellmann, "Iec 61131: A general overview and emerging trends," 2009.

[16] O. Castillo, P. Melin, and J. R. Castro, "Computational intelligence software for interval type-2 fuzzy logic," *Computer Applications in Engineering Education*, vol. 21, 2013.

[17] M. S. M. Aras, F. A. Ali, F. A. Azis, S. M. S. S. A. Hamid, and M. F. H. M. Basar, "Performances evaluation and comparison of two algorithms for fuzzy logic rice cooking system (matlab fuzzy logic toolbox and fuzzytech)," in *2011 IEEE Conference on Open Systems, ICOS 2011*, 2011.

[18] J. F. Rada-Vilela, "A fuzzy logic control library in c++," in *Proceedings of the Open Source Developers Conference, Auckland, New Zealand*, 2013, pp. 21–23.

[19] S. Guillaume and B. Charnomordic, "Fuzzy inference systems: An integrated modeling environment for collaboration between expert knowledge and data using fispro," *Expert Systems with Applications*, vol. 39, 2012.

[20] P. Cingolani and J. Alcalá-Fdez, "jfuzzylogic: A java library to design fuzzy logic controllers according to the standard for fuzzy control programming," *International Journal of Computational Intelligence Systems*, vol. 6, 2013.

[21] U. of Córdoba, "Jfml: Java fuzzy markup language," Online, 2025, accessed: 2025-03-25. [Online]. Available: http://www.uco.es/JFML/

[22] M. M. Rivera, E. Olvera-Gonzalez, and N. Escalante-García, "Upafuzzysystems: A python library for control and simulation with fuzzy inference systems," *Machines*, vol. 11, no. 5, p. 572, 2023. [Online]. Available: https://doi.org/10.3390/machines11050572

[23] D. Shiffman, *Learning Processing, Second Edition: A Beginner's Guide to Programming Images, Animation, and Interaction*, 2015.
[24] R. Kowalczyk, "On linguistic approximation of subnormal fuzzy sets," in *Annual Conference of the North American Fuzzy Information Processing Society - NAFIPS*, 1998.
[25] P. Mohindru, "Review on pid, fuzzy and hybrid fuzzy pid controllers for controlling non-linear dynamic behaviour of chemical plants," *Artificial Intelligence Review*, vol. 57, 2024.
[26] I. Baturone, F. J. Moreno-Velo, S. Sanchez-Solano, Ángel Barriga, P. Brox, A. A. Gersnoviez, and M. Brox, "Using xfuzzy environment for the whole design of fuzzy systems," in *IEEE International Conference on Fuzzy Systems*, 2007.
[27] E. Y. Bejarbaneh, A. Bagheri, B. Y. Bejarbaneh, S. Buyamin, and S. N. Chegini, "A new adjusting technique for pid type fuzzy logic controller using psoscalf optimization algorithm," *Applied Soft Computing Journal*, vol. 85, 2019.
[28] G. Prabhakar, S. Selvaperumal, and P. N. Pugazhenthi, "Fuzzy pd plus i control-based adaptive cruise control system in simulation and real-time environment," *IETE Journal of Research*, vol. 65, 2019.
[29] A. Lambora, K. Gupta, and K. Chopra, "Genetic algorithm- a literature review," in *Proceedings of the International Conference on Machine Learning, Big Data, Cloud and Parallel Computing: Trends, Prespectives and Prospects, COMITCon 2019*, 2019.
[30] J. M. Soto-Hidalgo, A. Vitiello, J. M. Alonso, G. Acampora, and J. Alcala-Fdez, "Design of fuzzy controllers for embedded systems with jfml," *International Journal of Computational Intelligence Systems*, vol. 12, 2019.